Module 03: Data Types and Constraints

Section 1: SQL Data Types

A **Data Type** is a rule that tells SQL what kind of value can be stored inside a column. When you create a table, you must select the correct data type for each column so the database knows how to store and handle the information properly.

1. Whole Numbers (INT, BIGINT)

These are used to store full numbers without any decimal points (like age, IDs, or quantities).

Data Type	Simple Definition & Usage	Exam Example
INT	Stores standard whole numbers. Ideal for everyday numbers like age, user IDs, or product stock count.	Age: 25, UserID: 4501
BIGINT	Stores extremely large whole numbers. Used when regular numbers are not big enough, such as global phone numbers or platform transactions.	Phone: 9860424538, Global Transactions

2. Decimal Numbers (DECIMAL, FLOAT, DOUBLE)

These are used when you need to store fractional numbers with decimal points (like price, weight, or percentage).

Data Type	Simple Definition & Usage	Exam Example
DECIMAL(p, s)	Stores exact numbers where accurate values are crucial. Perfect for money and prices because it never rounds numbers incorrectly.	Price: 45000.50 (e.g., DECIMAL(10,2))
FLOAT	Stores numbers with floating decimal points. It takes up less memory space but can round numbers slightly. Good for scientific readings.	Weight: 65.45
DOUBLE	Similar to FLOAT, but stores numbers with double the memory precision. It handles highly specific calculations safely.	GPS Coordinates: 19.3978, 72.8258

3. Text Strings (CHAR, VARCHAR, TEXT)

These are used to store characters, alphabets, words, or sentences.

Data Type	Simple Definition & Usage	Exam Example
CHAR(size)	Fixed length text. If you allocate 2 spaces, it always takes 2 spaces of memory, even if you fill only 1 space. Best for uniform codes.	State Codes: 'MH', 'DL'
VARCHAR(size)	Variable length text. It shrinks or grows based on the actual text you type. Saves huge memory space. Best for names or email IDs.	Names: 'Amit Sharma', Emails
TEXT	Stores very long blocks of words or descriptions. It does not have a strict pre-defined limit size constraint.	Product Reviews, Bio Notes

4. Date and Time (DATE, TIME, DATETIME)

These data types keep track of calendars, operational times, and transaction histories.

Data Type	Simple Definition & Usage	Exam Example
DATE	Stores only the calendar date format without any timestamp values attached. Format: YYYY-MM-DD.	Birth Date: '2005-09-20'
TIME	Stores only the hours, minutes, and seconds information. Format: HH:MM:SS.	Shift Start: '09:30:00'
DATETIME	Combines both values together to track the exact date and exact time a record was made. Format: YYYY-MM-DD HH:MM:SS.	Sale Timestamp: '2026-11-02 16:03:00'

Section 2: SQL Constraints

A **Constraint** is a control rule applied to a database column. It forces data integrity, ensuring that invalid, corrupted, or blank data values cannot be added to the tables during daily transactions.

The 7 Core Constraints:

Constraint Name	Simple Definition (Easy for Exams)	Real-World Purpose
PRIMARY KEY	Uniquely identifies each row in a table. It cannot contain duplicate values and can never be left empty.	Guarantees every Customer ID or Roll Number is one-of-a-kind.
FOREIGN KEY	Links a column in one table to the Primary Key of another table. It forms relationships between separate tables.	Ensures an Order row cannot be placed for a Customer ID that does not exist.
NOT NULL	Forces a column to never accept blank or empty values. A value must always be entered.	Prevents accounts from being saved without a Name or Email.
UNIQUE	Ensures all values stored inside a column are completely different from each other. No duplicates allowed.	Allows users to have unique Mobile Numbers or Aadhaar Cards.
DEFAULT	Sets a fixed backup value automatically if you forget to enter a value manually while creating a row.	Sets status to 'Active' or transaction country to 'India' by default.
CHECK	Validates data values against a small logical rule before allowing them to be saved. If the rule fails, the row is blocked.	Ensures Age is greater than 18 or Salary is greater than zero.
AUTO_INCREMENT	Tells SQL to automatically add +1 to the number for every new row you insert. You don't need to type the ID manually.	Generates sequential Invoice Numbers automatically (1, 2, 3, 4...).

Visual Diagram: Table Schema Blueprint

```
[ TABLE: students_database ]
================================================================================ |
Column Name | Data Type | Constraints Applied |
================================================================================ |
student_id | INT | PRIMARY KEY, AUTO_INCREMENT (Generates 1, 2, 3...) | | full_name |
VARCHAR(100) | NOT NULL (Cannot be empty) | | mobile_no | BIGINT | UNIQUE (No two students can
share a phone number) | | age | INT | CHECK (age >= 18) (Blocks under-age enrollment) | |
join_country | VARCHAR(50) | DEFAULT 'India' (Saves 'India' automatically) |
================================================================================
```

Standard SQL Syntax Example

Below is how data analysts read and structure table fields using the proper keyword declarations:

```
CREATE TABLE students_database (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    mobile_no BIGINT UNIQUE,
    age INT CHECK (age >= 18),
    join_country VARCHAR(50) DEFAULT 'India'
);
```

Section 3: Real-World Case Study

Scenario: You are working as a Junior Data Analyst at an e-commerce startup. The operations team is currently launching a premium subscription system. They built a basic database table to track user registration, but they are facing three severe errors:

1. Some rows have completely blank customer names, making it impossible to address shipping packages.
2. Users are accidentally registering the same phone number for multiple free trial accounts.
3. The application crashed yesterday because a customer typed a typo value of negative membership fees (e.g., Amount = -500).

Task: Design the corrected database table schema with precise data types and constraints to permanently fix these issues. Provide the solution query and explain why each constraint was selected.

The Analytical Solution Query

```
CREATE TABLE premium_subscriptions (
    subscription_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100) NOT NULL,
    phone_number BIGINT UNIQUE,
    fee_paid DECIMAL(10,2) CHECK (fee_paid >= 0),
    signup_date DATE DEFAULT (CURRENT_DATE)
);
```

Interview Logical Breakdown

Column & Rule	Analytical Justification for Interviews
subscription_id	Using **INT PRIMARY KEY AUTO_INCREMENT** creates a perfect unique tracking index that manages itself without manual input.
customer_name	Using **VARCHAR(100) NOT NULL** allows text names to fit dynamically while stopping anyone from saving a blank or empty user name.
phone_number	Using **BIGINT UNIQUE** easily holds long 10-digit Indian phone numbers while completely blocking duplicate registrations on the platform.
fee_paid	Using **DECIMAL(10,2) CHECK (fee_paid >= 0)** protects exact financial earnings with high precision and stops invalid negative values from corrupting company billing records.